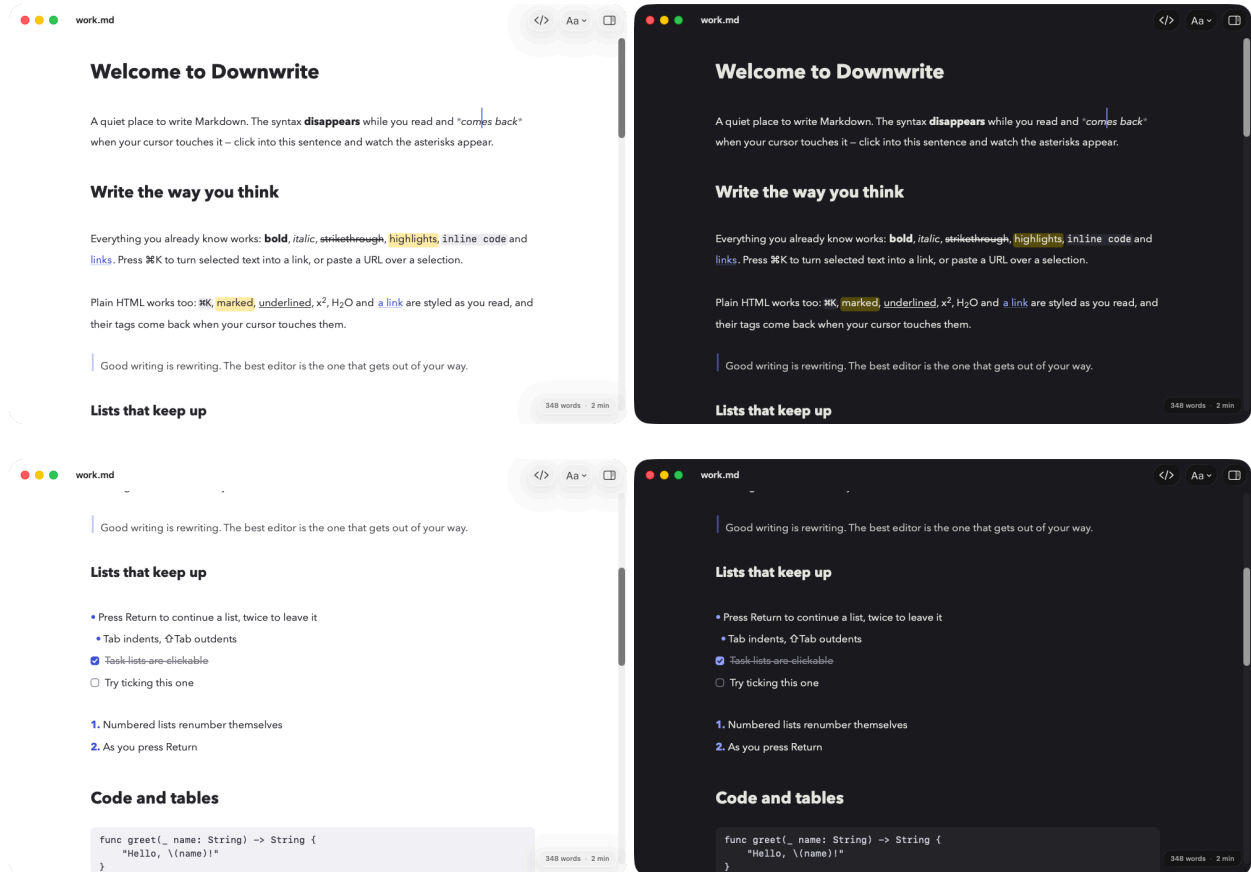




Downwrite

A quiet, native Markdown editor for the Mac — light-weight, beautiful to type in, and happy to be your default `.md` app.

Markdown syntax **disappears while you read** and **comes back when your cursor touches it**. Click into a bold word and the asterisks fade in; move away and they vanish. Everything is built with SwiftUI + AppKit, with no Electron, no web editor and no account.



work.md

⌘T

Insert a table

⌘O

Table of contents

</> Aa

Diagrams

Fenced `mermaid` blocks render as live diagrams. Click one to edit its source.

```
graph LR; Idea((Idea)) --> Write[Write in Markdown]; Write --> Happy{Happy?}; Happy -- no --> Write; Happy -- yes --> Share((Share))
```

Whole blocks of HTML are drawn too, under their source:

Centred, *styled* and laid out by the web engine.

348 words · 2 min

work.md

Edited

</> Aa



Downwrite

A quiet, native Markdown editor for the Mac – light-weight, beautiful to type in, and happy to be your default `.md` app.

Markdown syntax **disappears while you read** and **comes back when your cursor touches it**. Click into a bold word and the asterisks fade in; move away and they vanish. Everything is built with SwiftUI + AppKit, with no Electron, no web editor and no account.

3,395 words · 15 min

work.md

Contents

Welcome to Downwrite

Write the way you think

Lists that keep up

Code and tables

Diagrams

Write the way you think

A quiet place to write Markdown. The syntax **disappears** while you read and "comes back" when your cursor touches it – click into this sentence and watch the asterisks appear.

Write the way you think

Everything you already know works: **bold**, *italic*, ~~strikethrough~~, **highlights**, `inline code` and [links](#). Press **⌘K** to turn selected text into a link, or paste a URL over a selection.

Plain HTML works too: **⌘K**, **marked**, underlined, x², H₂O and a [link](#) are styled as you read, and their tags come back when your cursor touches them.

Good writing is rewriting. The best editor is the one that gets out of your way.

348 words · 2 min

work.md

Contents

Welcome to Downwrite

Write the way you think

Lists that keep up

Code and tables

Diagrams

Write the way you think

A quiet place to write Markdown. The syntax **disappears** while you read and "comes back" when your cursor touches it – click into this sentence and watch the asterisks appear.

Write the way you think

Everything you already know works: **bold**, *italic*, ~~strikethrough~~, **highlights**, `inline code` and [links](#). Press **⌘K** to turn selected text into a link, or paste a URL over a selection.

Plain HTML works too: **⌘K**, **marked**, underlined, x², H₂O and a [link](#) are styled as you read, and their tags come back when your cursor touches them.

Good writing is rewriting. The best editor is the one that gets out of your way.

348 words · 2 min

work.md

Contents

Welcome to Downwrite

Write the way you think

Lists that keep up

Code and tables

Diagrams

Write the way you think

A quiet place to write Markdown. The syntax **disappears** while you read and "comes back" when your cursor touches it – click into this sentence and watch the asterisks appear.

Write the way you think

Everything you already know works: **bold**, *italic*, ~~strikethrough~~, **highlights**, `inline code` and [links](#). Press **⌘K** to turn selected text into a link, or paste a URL over a selection.

Plain HTML works too: **⌘K**, **marked**, underlined, x², H₂O and a [link](#) are styled as you read, and their tags come back when your cursor touches them.

Good writing is rewriting. The best editor is the one that gets out of your way.

348 words · 2 min

work.md

Contents

Welcome to Downwrite

Write the way you think

Lists that keep up

Code and tables

Diagrams

Write the way you think

A quiet place to write Markdown. The syntax **disappears** while you read and "comes back" when your cursor touches it – click into this sentence and watch the asterisks appear.

Write the way you think

Everything you already know works: **bold**, *italic*, ~~strikethrough~~, **highlights**, `inline code` and [links](#). Press **⌘K** to turn selected text into a link, or paste a URL over a selection.

Plain HTML works too: **⌘K**, **marked**, underlined, x², H₂O and a [link](#) are styled as you read, and their tags come back when your cursor touches them.

Good writing is rewriting. The best editor is the one that gets out of your way.

348 words · 2 min

work.md

Contents

Welcome to Downwrite

Write the way you think

Lists that keep up

Code and tables

Diagrams

Write the way you think

A quiet place to write Markdown. The syntax **disappears** while you read and "comes back" when your cursor touches it – click into this sentence and watch the asterisks appear.

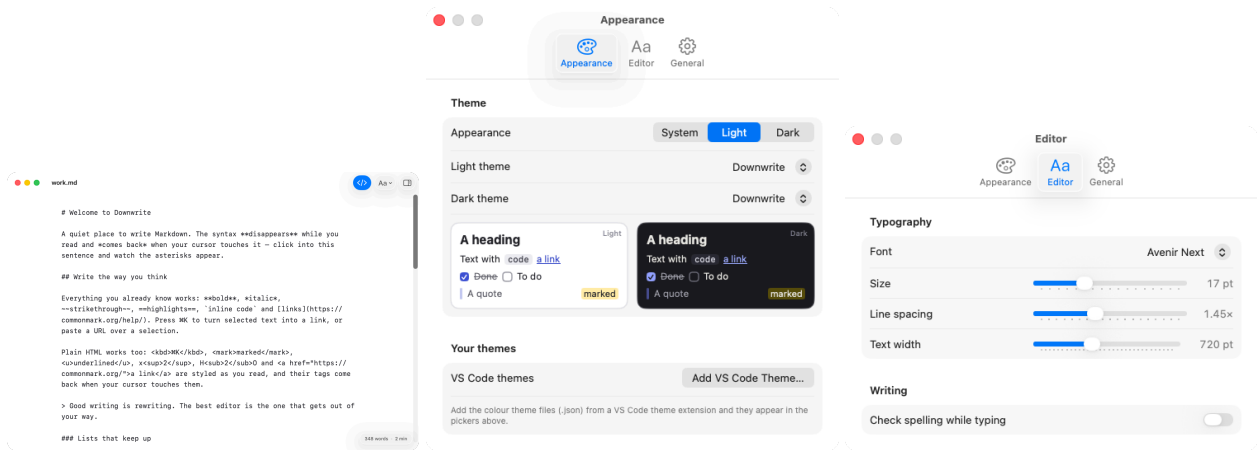
Write the way you think

Everything you already know works: **bold**, *italic*, ~~strikethrough~~, **highlights**, `inline code` and [links](#). Press **⌘K** to turn selected text into a link, or paste a URL over a selection.

Plain HTML works too: **⌘K**, **marked**, underlined, x², H₂O and a [link](#) are styled as you read, and their tags come back when your cursor touches them.

Good writing is rewriting. The best editor is the one that gets out of your way.

348 words · 2 min



Screenshots are captured automatically from the real app on a macOS 26 runner by the end-to-end self-test (see Testing).

Features

- **Hidden syntax, Bear-style.** `**bold**`, `*italic*`, ``code``, `[links](...)`, `# headings`, `> quotes`, code fences... all collapse to clean typography and re-expand around the caret.
- **CommonMark + GFM + extras.** Full CommonMark (via Apple's `swift-markdown` / `cmarmk-gfm`), plus tables, task lists (drawn as rounded checkboxes — click to tick; the `- [ ]` source shows when the caret enters it), strikethrough, autolinks, `==highlight==`, `[^footnotes]`, `$math$` styling and YAML front matter.
- **Editable tables.** Tables are shown as a real grid you type into, not as pipes and dashes. **Tab** jumps to the next cell (and adds a row from the last cell), **Return** moves down, arrows step across cell edges, `⇧Return` adds a line break. Drag the grips left of each row / above each column to reorder them, click a grip (or right-click any cell) for insert / delete / move / duplicate / align / sort, and use the `+` strips to append a row or column. Cell text keeps its Markdown (bold, links, code hide their syntax like everywhere else), pasting spreadsheet data fills several cells, and *Edit as Markdown* shows the source. The document text stays plain GFM, aligned and tidy.
- **Mermaid diagrams.** Fenced ````mermaid` blocks render as live diagrams (flowcharts, sequence, class, state, ER, Gantt, pie, journey, git graph, mind maps...). Mermaid is bundled, so it works offline. Click a diagram to edit its source; it re-renders as you type.
- **Table of contents.** Toggle a sidebar on the right (`⌘O`, the toolbar button, or `View ▶ Show Table of Contents`) that lists the document's headings, nested by level, with the section holding the caret highlighted. Click a heading and the editor scrolls it to the top and puts the caret there, ready to type. Headings show as they read in the editor (no `#`, `**` or link syntax), update as you type, and the sidebar remembers whether you left it open.

- **Source view.** Prefer to see the raw Markdown? The `</>` button in the toolbar (left of Format), or View ▶ Show Markdown Source (`⌘/`), switches the window to a plain monospaced editor: every character shown, no hidden syntax, no table grids, diagram cards or checkboxes. Your text, caret and undo history carry across, formatting shortcuts still work on the raw text, and the table-of-contents sidebar keeps following your headings. It is per window and starts off.
- **Follows the file on disk.** If another app (an editor, a sync client, `git checkout ...`) changes the file while it is open and you have no unsaved edits, Downwrite reloads it within about a second — or the moment you switch back to it — keeping your place: only the changed text is replaced, so the scroll position and caret stay put. (With unsaved edits, macOS's usual "changed by another application" prompt applies when you save.)
- **Folding.** Tuck away what you are not working on. A heading with content under it, and a list item with anything after its first paragraph, gets a chevron in the left margin when the pointer is on its line (and always while it is folded); click it, or the `...` chip that a folded line shows, to fold or unfold. A heading folds everything up to the next heading of the same or a higher level; an item folds its nested items, further paragraphs, code and tables. View ▶ **Fold / Unfold** (`⌘⌘← / ⌘⌘→`) act on the section around the caret (press Fold again to fold the next one out), **Fold All / Unfold All** (`⌘⌘⌘← / ⌘⌘⌘→`) on everything, and **Fold to Level** folds headings of one level and deeper. Folding only changes what is shown: your text, the file, the undo history and the "edited" mark never change, and folds stay with their heading as you type, undo or reload. The caret steps over folded text with the arrow and Page keys; Find, the contents sidebar and `#links` open whatever hides their target. Tables, diagrams, images and HTML cards inside a fold go with it. Folds belong to the window and are not saved: every file opens unfolded. The source view (`⌘/`) shows everything.
- **Keep on top.** Window ▶ Keep on Top (checkmarked while on) floats the current window above other windows, per window, until you turn it off.
- **Settings in three tabs.** `⌘,` opens **Appearance** (light / dark / system, the light and dark colour themes with previews, your imported VS Code themes), **Editor** (font, size, line spacing, text width, spell checking) and **General** (make Downwrite the default Markdown app, version, licences). It reopens on the tab you left it on.
- **Beautiful type.** Avenir Next (default), New York, SF Pro, Charter or SF Mono, with adjustable size, line spacing and column width. A centred readable column, rounded code cards, quote bars and soft rules.
- **Colour themes.** Settings → Appearance has a **Light theme** and a **Dark theme** picker, each with a live preview: *Downwrite* (the original, the default), *Catppuccin* (Latte for light, Mocha for dark) and *Radix* (the slate and indigo scales of [Radix Colors](#)). Which of the two shows follows the appearance setting below. Colour values are used under their MIT licences (see `THIRD-PARTY-NOTICES.md`).

- **Your own themes from VS Code.** Settings ▸ Appearance ▸ **Add VS Code Theme...** takes a VS Code colour theme file (the `.json` files in a theme extension's `themes` folder; comments and trailing commas are fine, and `include` d base themes are followed). Downwrite uses the colours that make sense for prose (page, text, links, accent from the heading colour, code, quotes, highlight, selection), works out the rest, and nudges any colour that would be hard to read. The theme is offered under *Light theme* or *Dark theme* by how light its page is, selected straight away, and kept in `~/Library/Application Support/Downwrite/Themes` (remove it from the same Settings pane).
- **Light, dark or follow macOS.** Settings → Theme (or View ▸ Appearance). The app icon is an Icon Composer asset (light, dark and tinted variants chosen by macOS) and the Dock icon also swaps between the light and dark artwork while the app runs.
- **Liquid Glass.** Built for macOS 26 "Tahoe" and ready for macOS 27 "Golden Gate": system toolbar and window chrome adopt Liquid Glass automatically, and the floating word-count pill uses `glassEffect`.
- **A proper Mac document app.** Tabs, Versions, autosave, Open Recent, drag-and-drop onto the Dock icon, Find (⌘F), full undo/redo. Opens and saves UTF-8 (± BOM), UTF-16 and legacy Windows-1252 files and **preserves line endings** (LF / CRLF) so it never rewrites a file's format behind your back.
- **Smart keys.** Return continues lists, tasks, numbered lists and quotes (press it twice to leave); Tab / ⇧Tab indent and outdent; paste a URL over selected text to make a link.
- **Small.** Under 10 MB on disk, most of it the bundled Mermaid engine. No network access needed except for remote images.
- **Inline images.** `![alt](path)` on its own line shows the picture below the (collapsed) Markdown.
- **Inline HTML.** HTML in a document is drawn, not shown as source. Tags inside a paragraph — `<kbd>`, `<b>` / `<strong>`, `<i>` / `<em>`, `<u>` / `<ins>`, `<s>` / `<del>`, `<mark>`, `<code>`, `<sub>`, `<sup>` and `<a href>` — are styled like their Markdown cousins and hide their tags until the caret touches them (`<a href>` links open with ⌘-click). Whole blocks (`<p align="center"><img ...></p>`, `<div>`, `<details>`, `<table>` ...) are rendered under their collapsed source in the editor's font and colours; click one to edit it and it re-renders as you type. Blocks are shown in a locked-down web view that never runs scripts, never leaves the page, and loads nothing but inline CSS and images (local ones from next to the document, or `https:`), so a downloaded file cannot do anything surprising. The source view (⌘/) shows the raw HTML.

Keyboard shortcuts

Action	Shortcut	Action	Shortcut
Bold	⌘B	Bulleted list	⇧⌘8
<i>Italic</i>	⌘I	Numbered list	⇧⌘7
Strikethrough	⇧⌘X	Task list	⇧⌘9
Highlight	⇧⌘H	Block quote	⇧⌘.
<code>Inline code</code>	⌘E	Code block	⌘⌘C
Link	⌘K	Horizontal rule	⌘⌘-
Heading 1–6	⌘1 ... ⌘6	Insert / format table	⌘⌘T / ⇧⌘⌘T
Body text	⌘0	Indent / outdent	⌘] / ⌘[(or Tab / ⇧Tab in lists)
Find	⌘F	Bigger / smaller text	⌘= / ⌘-
Table of contents	⇧⌘O	Markdown source view	⌘/
Next / previous table cell	Tab / ⇧Tab	New table row (in the last cell)	Tab or Return
Fold / unfold at the caret	⌘⌘← / ⌘⌘→	Fold all / unfold all	⇧⌘⌘← / ⇧⌘⌘→

Pressing a formatting shortcut again removes the formatting. With nothing selected it wraps the word under the caret (or inserts an empty pair). Standard shortcuts (⌘Z, ⇧⌘Z, ⌘C/V/X, ⌘A, ⌘S, ⌘W, ⌘T ...) work as in any Mac app. ⌘-click a link to open it.

Requirements

- **To run:** macOS 26 (Tahoe) or later — including macOS 27 (Golden Gate).
- **To build:** Xcode 26 or later (Swift 6.2+), installed as the toolchain — you never need to open it, everything builds from the Terminal. Command Line Tools alone are untested: `actool`, which compiles the adaptive light/dark app icon, ships only with full Xcode (without it `build-app.sh` falls back to a static icon).

Build and run

```
git clone https://github.com/derrickwheals/downwrite.git
cd downwrite

swift run Downwrite          # quick debug run (no bundle, no file
associations)
swift test                   # unit + integration tests (needs a GUI
session on macOS)
scripts/build-app.sh        # → dist/Downwrite.app (release build,
ad-hoc signed)
open dist/Downwrite.app
```

`scripts/build-app.sh` accepts `VERSION`, `BUILD`, `CONFIG` and `SIGN_IDENTITY` environment variables:

```
VERSION=1.2.0 BUILD=42 scripts/build-app.sh
```

Install locally

```
cp -R dist/Downwrite.app /Applications/
```

An ad-hoc signed build runs fine on the Mac that built it. If you copy it to another Mac, macOS will quarantine it: right-click ► **Open** once, or run `xattr -dr com.apple.quarantine /Applications/Downwrite.app`.

Make Downwrite your default Markdown editor

Any one of:

1. **Downwrite ► Make Downwrite the Default Markdown App** (or Settings ► Files) — macOS asks you to confirm.
2. In Finder, select any `.md` file ► **Get Info** (⌘I) ► **Open with** ► Downwrite ► **Change All...**
3. From the Terminal: Downwrite declares itself the *Owner* of `net.daringfireball.markdown` (`.md`, `.markdown`, `.mdown`, `.mkd`, `.mkdn`, `.mdwn`, `.mdtxt`, `.mdtext`) and an *Alternate* for plain text.

Deployment (distributable build)

To share Downwrite outside your own Mac you need an Apple Developer ID certificate.

```
# 1. Build, sign with hardened runtime + secure timestamp
SIGN_IDENTITY="Developer ID Application: Your Name (TEAMID)"
VERSION=0.9.0 BUILD=1 scripts/build-app.sh

# 2. Package
VERSION=0.9.0 scripts/make-dmg.sh          # → dist/Downwrite-0.9.0.dmg

# 3. Notarize and staple (one-time: xcrun notarytool store-credentials
"downwrite-notary" ...)
xcrun notarytool submit dist/Downwrite-0.9.0.dmg --keychain-profile
"downwrite-notary" --wait
xcrun stapler staple dist/Downwrite-0.9.0.dmg

# 4. Verify
spctl --assess --type open --context context:primary-signature -v
dist/Downwrite-0.9.0.dmg
```

The app is **not sandboxed** (it reads sibling files for relative links and opens documents anywhere you point it).

Mac App Store note. The GPL-3.0 and the App Store's terms are widely considered incompatible for *third parties* redistributing the code there. Direct download, GitHub Releases and Homebrew-style distribution are fine. As the sole copyright holder you may publish your own build elsewhere under different terms, but once you accept outside contributions you would need contributors' agreement (e.g. a CLA) to do so. If you ever go that route you would also add the App Sandbox entitlements to `Packaging/Downwrite.entitlements`.

Releases from GitHub

Pushing a tag like `v0.9.0` runs `.github/workflows/release.yml`, which builds on a `macos-26` runner, packages `Downwrite-<version>.dmg` and attaches it to a GitHub Release. If the repository has the secrets `DEVELOPER_ID_P12_BASE64`, `DEVELOPER_ID_P12_PASSWORD`, `DEVELOPER_ID_NAME`, `NOTARY_APPLE_ID`, `NOTARY_TEAM_ID` and `NOTARY_PASSWORD` (an app-specific password), the release is Developer-ID signed and notarized; otherwise it is ad-hoc signed.

Testing

Three layers, all run by CI on every push (see `.github/workflows/ci.yml`):

Layer	Where	What it covers
-------	-------	----------------

Core unit tests (Tests/DownwriteCoreTests)	Linux <i>and</i> macOS	The Markdown analysis engine (CommonMark/GFM structure, hidden-marker rules, Unicode/CRLF offsets, adversarial input), every formatting shortcut, list/quote continuation, the table model (parsing with cell ranges, every row/column command, Tab/Return navigation, sort, paste, column layout, leaving and deleting a table), file encodings and line endings, palette contrast (WCAG) and the white light-theme background, the table-of-contents outline (nesting, visible titles, caret → section), code-fence auto-close, task checkbox markers and toggling, the smallest-replacement text diff used when a file changes on disk, the colour themes (every built-in theme's legibility, Catppuccin/Radix values, fallbacks), the VS Code theme importer (JSON-with-comments, include , TextMate scope matching, hostile colours, and 300 random themes that must all stay readable), Mermaid helpers.
App integration tests (Tests/DownwriteTests)	macOS	Real <code>NSTextView</code> editor in a real window: attributes for every Markdown construct, markers hiding/revealing as the caret moves, typing, undo/redo, smart Return/Tab, paste-to-link, theme switching, Mermaid rendering of ten diagram types plus error handling, the table of contents sidebar's model, caret tracking

and click-to-scroll, the table grid (layout over the collapsed source, typing with one-step undo, Tab/Return/arrow navigation, the right-click menu, grip dragging, focus hand-off, Markdown-source view), pixel checks of the drawing (a selection stays visible inside code, inline-code pills hug the text, checkboxes are filled or outlined), task checkboxes (hidden source, click-to-toggle without moving the caret, empty items keep a normal line), empty headings and quote markers keeping their height, fence auto-close, HTML cards (sized to their content, scripts never run, navigation refused, local images embedded, reflow on a narrower column), inline HTML tags, the source view, folding (regions in `DownwriteCore`, the fold state machine and its mapping through edits, hidden lines taking no room, tables, diagrams and cards going with a fold, the caret stepping over folds, Return/Backspace/Delete at a fold's edge, the chevron and chip drawn in every theme, speed on 10,000 lines), Keep on Top, the editor following a file that changes on disk (caret and scroll kept, undo dropped), theme selection per appearance, imported themes (store, replace, remove, restart), Info.plist file-association checks, and PNG snapshots for visual review.

End-to-end self-test (Downwrite --selftest)	macOS	Launches the <i>packaged app</i>, opens a file through the document system, presses real ⌘B/⌘I/⌘E/⌘2/⌘K key events through the real menu bar, types, saves to disk and re-reads the file, waits for a Mermaid card, tabs through the table grid with real key events (adding a row, typing, the right-click menu, undo), toggles the table-of-contents sidebar with ^⌘O and clicks a heading to scroll to it, lets a second process rewrite the open file (appending, then replacing it atomically) and checks the editor follows, clicks a drawn checkbox with a real mouse event, presses Return after a task, switches to the source view with ⌘/ and back, chooses <i>Keep on Top</i> from the Window menu, selects every built-in theme and imports a VS Code theme file, folds and unfolds a real document with the menu shortcuts, the mouse, Find and Return/Delete and checks the saved file is byte-identical, flips light → dark → system theme, and screenshots the actual window and the Settings window.
--	--------------	--

Run the core tests on any machine with Docker (no Mac needed):

```
scripts/linux-test.sh
```

Run the end-to-end test yourself:

```

scripts/build-app.sh
dist/Downwrite.app/Contents/MacOS/Downwrite --selftest-
out=/tmp/downwrite-e2e --selftest-
input="$PWD/Sources/Downwrite/Resources/Welcome.md"
cat /tmp/downwrite-e2e/selftest-report.txt

```

Project layout

```

Sources/DownwriteCore/    Pure Swift (Foundation + swift-markdown):
analysis, commands, palettes, file coding
Sources/Downwrite/        macOS app: SwiftUI shell, TextKit 1 editor,
styler, Mermaid renderer, settings
Tests/DownwriteCoreTests  Linux + macOS unit tests
Tests/DownwriteTests      macOS integration tests
Packaging/                Info.plist, entitlements, icon artwork
scripts/                  build-app.sh, make-dmg.sh, linux-test.sh, ci-
local.sh, generate-icons.py
ThirdParty/               Third-party licence texts (swift-markdown,
cmark-gfm, Mermaid, Catppuccin, Radix Colors)
docs/                     Backlog notes, plus screenshots and CI
reports written by the end-to-end test
LICENSE                   GNU GPL v3.0

```

How it works, in one paragraph: `MarkdownAnalyzer` parses the text with `cmark-gfm` and produces an immutable `MarkdownAnalysis` — style spans, *marker* ranges (the syntax characters) with the range that reveals each one, and per-line block styles.

`MarkdownStyler` turns that into `NSTextStorage` attributes; hidden markers get a 0.1 pt clear font so they stay in the text (copy, undo and find keep working) but take no space.

When the caret moves, only the lines whose markers changed state are restyled. The table-of-contents sidebar is a SwiftUI `.inspector` fed by `TableOfContents` (rows built from the analysed headings, with `MarkdownAnalysis.headingIndex(at:)` mapping the caret to its section); a click moves the caret to the heading and

`EditorTextView.scrollToTop(of:)` scrolls it into place. Top-level tables are parsed into a `TableModel` (cell source text, alignments, where each cell sits in the document) and shown by `TableGridView`, a grid of small TextKit 1 editors laid over the table's collapsed source; every edit goes back through `TableModel.markdown` so the document text is the single source of truth and undo just works. Mermaid blocks are rendered once in a hidden `WKWebView` to SVG and shown in a click-through card placed in vertical space reserved under the collapsed source; top-level HTML blocks use the same card, with the document's HTML in a script-less, navigation-less `WKWebView` page

built by `HTMLSupport` (content-security policy first) and measured to reserve exactly its height.

A few more pieces, all kept in the pure-Swift core so they are tested on Linux:

`FoldRegion` / `FoldState` decide what can fold and what is hidden, and carry folds through every edit; task items (`- [ ]`) are one hideable marker that the layout manager replaces with a drawn checkbox; `FenceEditing` closes a code fence as you type it; `TextDiff` turns "the file changed on disk" into the smallest replacement so the view keeps its place; `ThemeCatalog` / `ThemeLibrary` hold the colour themes, and `VSCoTheme` converts a VS Code theme file into one (with a contrast guard so an odd theme can never be unreadable). The source view is the same editor with every overlay and style switched off.

Known limitations

- Images are previewed inline when they sit on a line of their own (local paths are resolved relative to the document; `http(s)` images load from the network). Images inside a paragraph are styled but not previewed.
- Math ($\$...$, $\$ \$... \$ \$$) is styled, not typeset.$
- HTML: only top-level blocks are rendered (HTML inside a quote or list item stays source), links inside a rendered block are not clickable, and `<details>` that wraps Markdown is drawn piece by piece because Markdown ends an HTML block at the first blank line. Inline tags other than the ones listed above (`<br>` , `<span style=...>` , `<font>` , `<small>` ...), unclosed tags and tags that cross a Markdown span stay as dim source.
- Folds are kept in memory per window (nothing is written to the file or to disk), and Replace All is one change of the text, so a fold whose heading lies between the first and the last match is closed. Only top-level headings fold (not those inside quotes or list items).
- Numbered tasks (1. []) keep their number and the raw [] ; only bulleted tasks become checkboxes. The source view is deliberately plain (no syntax colouring).
- VS Code theme import reads colours only: `.tmTheme` token files, `.vsix` packages, semantic token colours and theme fonts are not used (unsupported parts are reported, not fatal). Themes are sorted into light or dark by their page colour, whatever the file's `type` says.
- Files are checked for outside changes once a second. A document with unsaved edits is not reloaded behind your back; macOS's own "changed by another application" prompt handles that when you save.
- Tables inside block quotes or list items stay as Markdown source (only top-level tables become a grid). Column widths are automatic — Markdown has nowhere to store them — and cells cannot hold real line breaks (use `<br>` , which ↵Return inserts).
- Liquid Glass chrome comes from the system on macOS 26+; Downwrite deliberately keeps the writing surface opaque for legibility. Icon Composer light/dark/tinted variants are compiled with `actool` when it succeeds in `scripts/build-app.sh` ; otherwise the app falls back to the bundled `.icns` (the Dock icon still switches light/dark at runtime).
- Built and tested against the macOS 26 SDK (Xcode 26.x). macOS 27 specific APIs are not required.

License

Downwrite is free software: you can redistribute it and/or modify it under the terms of the **GNU General Public License v3.0** — see [LICENSE](#) . Copyright © 2026 Derrick Wheals.

Third-party components (swift-markdown, cmark-gfm, Mermaid, and the Catppuccin and Radix colour values) and their licences are listed in [THIRD-PARTY-NOTICES.md](#) ; the licence texts ship inside the app at `Contents/Resources/Legal/` . Contributions are accepted under the same licence.

Credits

Markdown parsing by [swift-markdown](#) (cmark-gfm). Diagrams by [Mermaid](#) (MIT, see `ThirdParty/mermaid-LICENSE.txt`). Colour themes: [Catppuccin](#) (Latte and Mocha, MIT) and [Radix Colors](#) (MIT); licence texts in `ThirdParty/` .